

JOB AGENT

Complete Architecture Analysis

Generated: April 14, 2026 | 09:37

Summary: A production-ready, multi-layer agentic job application automation system combining FastAPI, LangGraph, Playwright browser automation, MongoDB vector store, and intelligent LLM integration.

TABLE OF CONTENTS

- 1. Overall Architecture
- 2. Layered Architecture
- 3. LangGraph Workflow State Machine
- 4. Vector Store & Knowledge Retrieval
- 5. When & Where Agents Are Called
- 6. Key Decision Points
- 7. Value Resolution Strategy
- 8. Data Flow Diagram
- 9. Platform-Specific Handlers
- 10. Chrome Extension Mode
- 11. State Persistence
- 12. Component Summary Table

1. OVERALL ARCHITECTURE

This is a **multi-layer agentic job application automation system** combining:

- FastAPI backend for REST/WebSocket APIs
- LangGraph state machine for orchestrating application workflows
- Playwright browser automation for form interaction
- MongoDB vector store for resume knowledge retrieval
- Chrome extension for in-page filling
- LLM integration (Groq) for intelligent field resolution

Core Philosophy: Break job applications into structured steps, retrieve relevant resume data, fill intelligently with human fallback for edge cases.

2. LAYERED ARCHITECTURE

2.1 Layer 1: User Interface

Web UI (ui/app.js, ui/index.html):

- Resume upload via /api/profile/resume
- WebSocket connection for real-time application progress
- Manual field answers when agent needs clarification

Chrome Extension (chrome_extension/):

- Manifest V3 with content script injection
- Runs on host pages (http://*/*, https://*/*)
- Sends field arrays to /api/extension/resolve endpoint
- Returns fill plan with fills, blocked, and skipped fields

2.2 Layer 2: API Layer (FastAPI)

@app.post("/api/profile/resume"): Upload resume to MongoDB/GridFS

@app.get("/api/profile/resume"): Get resume metadata for profile

@app.post("/api/extension/resolve"): Extension field resolution

@app.ws("/ws/stream"): WebSocket for application streaming and real-time updates

Resume processing: Chunks text into 700-character segments with 120-character overlap, generates embeddings using Sentence-Transformers or lightweight hash-based fallback.

2.3 Layer 3: Core Agent Orchestration

JobAgent class (job_agent/agent/job_agent.py): Entry point for workflow execution. Maintains active_runner, active_application_id, and active_profile_id. Key methods: run(), continue_run(), apply().

ApplicationGraphRunner class (job_agent/graph/runner.py): Compiles LangGraph state machine (or fallback if LangGraph not installed). Manages browser instance and Playwright tools. Builds initial state with profile/application/platform data. Controls graph execution loop.

3. LANGGRAPH WORKFLOW - THE STATE MACHINE

The **9-node LangGraph workflow** processes each application step-by-step:

bootstrap → read_page → plan_fields → retrieve_profile_context

→ fill_fields → evaluate_page → assess_gaps

→ [human_pause OR advance_or_submit] → [loop or END]

Node Responsibilities:

Node	Purpose	Key Actions
bootstrap	Initialize session	Opens job page, creates application_id, validates URL
read_page	Extract form	snapshot_form(), detects fields with Playwright
plan_fields	Classify fields	Uses field_classifier.classify() to map labels → field types
retrieve_profile_context	Find resume data	Searches resume chunks by embedding similarity
fill_fields	Populate fields	fill_text(), select_option(), upload_resume() tools
evaluate_page	Read page state	Gets fresh snapshot to detect form changes/errors
assess_gaps	Identify unfilled fields	Creates HumanEscalation objects for low-confidence fields
human_pause	Get human feedback	Sends question via WebSocket, waits for response
advance_or_submit	Navigate form	Clicks Next button or submits application

4. VECTOR STORE & KNOWLEDGE RETRIEVAL SYSTEM

4.1 Architecture

Resume Processing Pipeline:

1. `extract_resume_text()` - Parse PDF/DOCX/RTF/TXT files
2. `chunk_resume_text()` - Split into 700-char chunks with 120-char overlap
3. For each chunk:
 - `generate_embedding()` - Sentence-Transformers or hash-based
 - `_infer_field_types()` - Detect relevant fields via keyword hints
 - `_extract_entities()` - Find proper nouns (company names, titles)
 - `_extract_date_range()` - Parse employment dates
4. Store in MongoDB `resume_chunks` collection with:
 - `profile_id`, `chunk_text`, embedding vector
 - `field_types`, `entities`, `date_range`, `confidence`

4.2 Retrieval Methods

Vector Similarity Search: Generates query embedding from field label, scans resume chunks, calculates cosine similarity, returns top matches with confidence scores > 0.6.

Learned Answers: Stores Q&A; pairs in `answers_collection` with embeddings. When similar question asked, retrieves exact previous answer. Used for dynamic/profile-specific fields (notice period, portfolio links).

Dual-Mode Embedding:

- Transformer Mode: `SentenceTransformer('all-MiniLM-L6-v2')` - 384 dimensions
- Hash Embedding: SHA256-based, 192 dimensions - works offline as fallback

5. WHEN & WHERE AGENTS ARE CALLED

5.1 Entry Point 1: Web UI WebSocket

Client connects via WebSocket and sends: `{"type": "start", "url": "...", "profile_id": "..."}`

Server creates JobAgent instance, calls agent.run() with URL, WebSocket reference, and profile_id. Real-time status updates flow back through WebSocket.

5.2 Entry Point 2: Chrome Extension

Extension injects content script, extracts form fields, POSTs to **/api/extension/resolve** with field list. Returns fill plan with 'fills', 'blocked', and 'skipped' fields. Extension injects fills directly into page DOM.

5.3 Agent Invocation Flow

1. User triggers application (Web UI or Extension)
2. JobAgent.run() called
 - Detects platform (Workday? Greenhouse?)
 - Creates ApplicationGraphRunner
 - Builds initial state
3. LangGraph executes nodes sequentially
 - Each node can trigger LLM calls, resume searches, browser tools
4. If gaps detected: send HumanEscalation via WebSocket
 - Pause application
 - Receive human answer
 - Resume from human_pause node
5. Persist state to MongoDB
 - Save field values, track progress
 - Allow continuation later

6. KEY DECISION POINTS (CONDITIONAL EDGES)

Node	Condition	Next Node
assess_gaps	stop_reason == 'no_fields_detected'	END
assess_gaps	pending_questions exist	human_pause
assess_gaps	All fields filled well	advance_or_submit
human_pause	User stopped session	END
human_pause	User provided answer	fill_fields (retry)
advance_or_submit	continue_loop == True	read_page (more steps)
advance_or_submit	Form complete	END

7. VALUE RESOLUTION STRATEGY

When the **retrieve_profile_context** node runs, it calls `resolve_field()` which tries in order:

- **1. Direct Profile Lookup:** first_name, email, phone stored in profile JSON
- **2. Resume RAG:** Semantic search on resume chunks for company_name, job_title, skills, location, etc.
- Returns confidence score
- **3. Learned Answers:** Semantic search in previous Q&A; for dynamic fields (portfolio, linkedin, github, notice_period)
- **4. Default Values:** phone_device_type='Mobile', job_source='LinkedIn'
- **5. Human Escalation:** If confidence < 0.6 OR required field is empty
- Creates HumanEscalation → WebSocket → human input

This multi-tier approach ensures high accuracy while minimizing human interruptions.

8. DATA FLOW DIAGRAM - SINGLE APPLICATION RUN

STAGE 1: INITIALIZATION

- User provides job URL + profile_id
- Platform detected (Workday/Greenhouse/etc)
- Resume chunks retrieved from DB

STAGE 2: FORM EXTRACTION & PLANNING

- Playwright opens job page
- Clicks 'Apply' button
- Parses form fields with selectors
- Classifies each field type
- Sends 'Detected field: First Name' → UI

STAGE 3: VALUE RETRIEVAL & FILLING

- For each field: search resume embeddings
- Semantic search learned answers
- Get confidence score
- If confident: Auto-fill
- If uncertain: Queue for human review

STAGE 4: GAP ASSESSMENT

- Collect unfilled & low-confidence fields
- Create HumanEscalation list
- Send via WebSocket: 'Questions need review'

STAGE 5: HUMAN HANDOFF (Optional)

- Display questions in UI
- User provides answers

- Store as learned answers + embedding
- Resume from human_pause node

STAGE 6: FORM ADVANCEMENT

- Check for hidden validation errors
- Click 'Next' or 'Submit' button
- Wait for form ready
- Repeat for multi-step forms

STAGE 7: PERSISTENCE

- Save application result to MongoDB
- Track steps, fields filled, questions asked
- Allow user to continue later
- Return summary to client

9. PLATFORM-SPECIFIC HANDLERS

Located in **job_agent/handlers/**, each platform gets specialized handling:

- `workday_handler.py` - Workday-specific validation, auth gates
- `greenhouse_handler.py` - Greenhouse form parsing quirks
- `lever_handler.py` - Lever's JavaScript-based forms
- `smartrecruiters_handler.py` - Custom field mappings
- `ashby_handler.py`, `bamboo_handler.py`, `icims_handler.py`, `jobvite_handler.py`
- `successfactors_handler.py`, `taleo_handler.py` - Various ATS platforms
- `base_handler.py` - Generic platform fallback

detect_platform() in `agent/platform_detector.py` routes to specialized handler based on URL domain.

10. CHROME EXTENSION MODE

Instead of Playwright opening a separate browser, the chrome extension offers an alternative flow:

1. Extension runs content script in current tab
2. Extracts field arrays via `document.querySelectorAll()`
3. POSTs to `/api/extension/resolve` with field list
4. Returns fill plan
5. Extension injects fills directly into page DOM

Advantage: User stays in their flow, no separate popup browser window required.

Technology: Manifest V3 with content script injection, background service worker, and popup UI for configuration.

11. STATE PERSISTENCE

Every 2-3 node executions, the complete application state is saved to MongoDB via **`_persist()`**:

The state document contains: `profile_id`, `fields`, `resolved_values`, `pending_questions`, `knowledge_hits`, `submitted status`, `application_id`, `timestamps`, and `browser mode`.

Benefits:

- Resuming interrupted applications via `JobAgent.continue_run()`
- Audit trail of all steps taken
- Fallback for browser crashes
- Multi-session support

Database Fallback: MongoDB attempts; if unavailable, in-memory storage used during development.

12. COMPONENT SUMMARY TABLE

Component	Purpose	When Used
Vector Store	Resume embeddings + semantic search	During retrieve_profile_context node
LangGraph Nodes	Sequential workflow orchestration	From job URL to submit
LLM (Groq)	Parse ambiguous fields, generate and correct	On demand for complex fields
Platform Detector	Route to specialized handler	At workflow bootstrap
Field Classifier	Map label → field type	In plan_fields node
WebSocket	Bidirectional real-time communication	Progress updates + human Q&A
Playwright	Form automation	fill_fields, advance_or_submit nodes
MongoDB	Persistent state + knowledge base	Every node + resume storage
Chrome Extension	In-page filling	Alternative to Playwright mode

Summary: This architecture is modular, extensible, and production-ready with graceful fallbacks for when heavy components (Mongo, Ollama, Transformers) are unavailable.